

Aug 7, 2025

GPT-5 prompting guide

This recipe is archived and may reference outdated models or APIs.



Anoop Kotha (OpenAI) , Julian Lee (OpenAI) , Eric Zakariasson ,
Erin Kavanaugh (OpenAI)

[View on GitHub](#)[Download raw](#)

GPT-5, our newest flagship model, represents a substantial leap forward in agentic task performance, coding, raw intelligence, and steerability.

While we trust it will perform excellently “out of the box” across a wide range of domains, in this guide we’ll cover prompting tips to maximize the quality of model outputs, derived from our experience training and applying the model to real-world tasks. We discuss concepts like improving agentic task performance, ensuring instruction adherence, making use of newly API features, and optimizing coding for frontend and software engineering tasks - with key insights into AI code editor Cursor’s prompt tuning work with GPT-5.

We’ve seen significant gains from applying these best practices and adopting our canonical tools whenever possible, and we hope that this guide, along with the [prompt optimizer tool](#) we’ve built, will serve as a launchpad for your use of GPT-5. But, as always, remember that prompting is not a one-size-fits-all exercise - we encourage you to run experiments and iterate on the foundation offered here to find the best solution for your problem.

Agentic workflow predictability

We trained GPT-5 with developers in mind: we’ve focused on improving tool calling, instruction following, and long-context understanding to serve as the best foundation model for agentic applications. If adopting GPT-5 for agentic and tool calling flows, we recommend upgrading to the [Responses API](#), where reasoning is persisted between tool calls, leading to more efficient and intelligent outputs.

[Ask AI](#)

Loading docs agent...

Method:

- Start broad, then fan out to focused subqueries.
- In parallel, launch varied queries; read top hits per query. Deduplicate paths and cache; don't repeat queries.
- Avoid over searching for context. If needed, run targeted searches in one parallel batch.

Early stop criteria:

- You can name exact content to change.
- Top hits converge (~70%) on one area/path.

Escalate once:

- If signals conflict or scope is fuzzy, run one refined parallel batch, then proceed.

Depth:

- Trace only symbols you'll modify or whose contracts you rely

Loading docs agent...

Prompting for more eagerness

On the other hand, if you'd like to encourage model autonomy, increase tool-calling persistence, and reduce occurrences of clarifying questions or otherwise handing back to the user, we recommend increasing `reasoning_effort`, and using a prompt like the following to encourage persistence and thorough task completion:

`<persistence>`

- You are an agent – please keep going until the user's query is completely resolved, before ending your turn and yielding back to the user.
- Only terminate your turn when you are sure that the problem is solved.
- Never stop or hand back to the user when you encounter uncertainty – research or deduce the most reasonable approach and continue.
- Do not ask the human to confirm or clarify assumptions, as you



Loading docs agent...

...always begin by representing the user's goal in a friendly, clear, and concise manner, before calling any tools.

- Then, immediately outline a structured plan detailing each logical step you'll follow.
- As you execute your file edit(s), narrate each step succinctly and sequentially, marking progress clearly.
- Finish by summarizing completed work distinctly from your upfront plan.

</tool_preambles>

Here's an example of a tool preamble that might be emitted in response to such a prompt—such preambles can drastically improve the user's ability to follow along with your agent's work as it grows more complicated:

```
"output": [  
  {  
    "id":
```



Loading docs agent...

```
CA\","unit\":"f\"},"",
  "call_id": "call_X0nF4B9DvB8EJVB3JvWnGg83",
  "name": "get_weather"
},
],
```

Reasoning effort

We provide a `reasoning_effort` parameter to control how hard the model thinks and how willingly it calls tools; the default is `medium`, but you should scale up or down depending on the difficulty of your task. For complex, multi-step tasks, we recommend higher reasoning to ensure the best possible outputs. Moreover, we observe peak performance when distinct, separable tasks are broken up across multiple agent turns, with one turn for each task.

Loading docs agent...

implementation abilities. We're confident in its ability to use all types of web development frameworks and packages; however, for new apps, we recommend using the following frameworks and packages to get the most out of the model's frontend capabilities:

- Frameworks: Next.js (TypeScript), React, HTML
- Styling / UI: Tailwind CSS, shadcn/ui, Radix Themes
- Icons: Material Symbols, Heroicons, Lucide
- Animation: Motion
- Fonts: San Serif, Inter, Geist, Mona Sans, IBM Plex Sans, Manrope

Zero-to-one app generation

GPT-5 is excellent at building applications in one shot. In early experimentation with the model, users have found that prompts like the one below—asking the model to

Loading docs agent...

need to change the actual content of the rules according to your programming design taste!

<code_editing_rules>



<guiding_principles>

- Clarity and Reuse: Every component and page should be modular and reusable. Avoid duplication by factoring repeated UI patterns into components.
- Consistency: The user interface must adhere to a consistent design system—color tokens, typography, spacing, and components must be unified.
- Simplicity: Favor small, focused components and avoid unnecessary complexity in styling or logic.
- Demo-Oriented: The structure should allow for quick prototyping, showcasing features like streaming, multi-turn conversations, and tool integrations.
- Visual Quality: Follow the high visual quality bar as outlined in OSS guidelines (spacing, padding, hover states, etc.)

Loading docs agent...

```
- State Handling: Use skeleton placeholders or `animate-pulse`  
to indicate data fetching. Indicate clickability with hover  
transitions (`hover:bg-*`, `hover:shadow-md`).  
- Accessibility: Use semantic HTML and ARIA roles where  
appropriate. Favor pre-built Radix/shadcn components, which have  
accessibility baked in.  
</ui_ux_best_practices>  
  
<code_editing_rules>
```

Collaborative coding in production: Cursor's GPT-5 prompt tuning

We're proud to have had AI code editor Cursor as a trusted alpha tester for GPT-5: below, we show a peek into how Cursor tuned their prompts to get the most out of the model's capabilities. For more information, their team has also published a blog

Loading docs agent...

efficient, concise status updates and final work summary with much more readable code diffs.

Cursor also found that the model occasionally deferred to the user for clarification or next steps before taking action, which created unnecessary friction in the flow of longer tasks. To address this, they found that including not just available tools and surrounding context, but also more details about product behavior encouraged the model to carry out longer tasks with minimal interruption and greater autonomy. Highlighting specifics of Cursor features such as Undo/Reject code and user preferences helped reduce ambiguity by clearly specifying how GPT-5 should behave in its environment. For longer horizon tasks, they found this prompt improved performance:

Be aware that the code edits you make will be displayed to the user as proposed changes, which means (a) your code edits can be



Loading docs agent...

softening the language around thoroughness. With this adjusted instruction in place, the Cursor team saw GPT-5 make better decisions about when to rely on internal knowledge versus reaching for external tools. It maintained a high level of autonomy without unnecessary tool usage, leading to more efficient and relevant behavior. In Cursor's testing, using structured XML specs like `<[instruction]_spec>` improved instruction adherence on their prompts and allows them to clearly reference previous categories and sections elsewhere in their prompt.

`<context_understanding>`



...

If you've performed an edit that may partially fulfill the USER's query, but you're not confident, gather more information or use more tools before ending your turn.

Bias towards not asking the user for help if you can find the

Loading docs agent...

overrides in the prompt for specific contexts where you might want the model to deviate from the global default. Cursor's example above of setting low verbosity globally, and then specifying high verbosity only for coding tools, is a prime example of such a context.

Instruction following

Like GPT-4.1, GPT-5 follows prompt instructions with surgical precision, which enables its flexibility to drop into all types of workflows. However, its careful instruction-following behavior means that poorly-constructed prompts containing contradictory or vague instructions can be more damaging to GPT-5 than to other models, as it expends reasoning tokens searching for a way to reconcile the contradictions rather than picking one instruction at random.

Loading docs agent...

Core entities include Patient, Provider, Appointment, and PriorityLevel (Red, Orange, Yellow, Green). Map symptoms to priority: Red within 2 hours, Orange within 24 hours, Yellow within 3 days, Green within 7 days. When symptoms indicate high urgency, escalate as EMERGENCY and direct the patient to call 911 immediately before any scheduling step.

Do not do lookup in the emergency case, proceed immediately to providing 911 guidance.

- Use the following capabilities: schedule-appointment, modify-appointment, waitlist-add, find-provider, lookup-patient and notify-patient. Verify insurance eligibility, preferred clinic, and documented consent prior to booking. Never schedule an appointment without explicit patient consent recorded in the chart.

- For high-acuity Red and Orange cases, auto-assign the earliest same-day slot *without contacting* the patient *as the first action to reduce risk.* If a suitable provider is unavailable,

Loading docs agent...

types of issues.

Minimal reasoning

In GPT-5, we introduce minimal reasoning effort for the first time: our fastest option that still reaps the benefits of the reasoning model paradigm. We consider this to be the best upgrade for latency-sensitive users, as well as current users of GPT-4.1.

Perhaps unsurprisingly, we recommend prompting patterns that are similar to [GPT-4.1 for best results](#). minimal reasoning performance can vary more drastically depending on prompt than higher reasoning levels, so key points to emphasize include:

1. Prompting the model to give a brief explanation summarizing its thought process at the start of the final answer, for example via a bullet point list, improves performance on tasks requiring higher intelligence.

Loading docs agent...

By default, GPT-5 in the API does not format its final answers in Markdown, in order to preserve maximum compatibility with developers whose applications may not support Markdown rendering. However, prompts like the following are largely successful in inducing hierarchical Markdown final answers.

- Use Markdown **only** where semantically correct* (e.g., ``inline code``, ````code fences````, lists, tables).
- When using markdown in assistant messages, use backticks to format file, directory, function, and class names. Use `\(` and `\)` for inline math, `\[` and `\]` for block math.



Occasionally, adherence to Markdown instructions specified in the system prompt can degrade over the course of a long conversation. In the event that you experience

Loading docs agent...

SWE-Bench verified developer instructions

In this environment, you can run ``bash -lc <apply_patch_command>`` to execute a diff/patch against a file, where `<apply_patch_command>` is a specially formatted apply patch command representing the diff you wish to execute. A valid `<apply_patch_command>` looks like:



```
apply_patch << 'PATCH'
*** Begin Patch
[YOUR_PATCH]
*** End Patch
PATCH
```

Where `[YOUR_PATCH]` is the actual content of your patch.

Loading docs agent...

```
query: string, // default: null
path?: string, // default: ""
max_results?: number, // default: 50
}) => any;
```

Set 2: 2 functions, terminal-native

```
type run = (_: {
  command: string[], // default: null
  session_id?: string | null, // default: null
  working_dir?: string | null, // default: null
  ms_timeout?: number | null, // default: null
  environment?: object | null, // default: null
  run_as_user?: string | null, // default: null
}) => any;
```

```
type send_input = (_: {
  session_id: string, // default: null
  text: string, // default: null
```

Loading docs agent...

addition, ensure function calls have the correct arguments.

Workflow steps

- At the beginning of the conversation, you have to authenticate the user identity by locating their user id via email, or via name + zip code. This has to be done even when the user already provides the user id.
- Once the user has been authenticated, you can provide the user with information about order, product, profile information, e.g. help the user look up order id.
- You can only help one user per conversation (but you can handle multiple requests from the same user), and must deny any requests for tasks related to any other user.
- Before taking consequential actions that update the database (cancel, modify, return, exchange), you have to list the action detail and obtain explicit user confirmation (yes) to proceed.
- You should not make up any information or knowledge or procedures not provided from the user or the tools, or give

Loading docs agent...

- The user needs to confirm the order id and the reason (either 'no longer needed' or 'ordered by mistake') for cancellation.
- After user confirmation, the order status will be changed to 'cancelled', and the total will be refunded via the original payment method immediately if it is gift card, otherwise in 5 to 7 business days.

Modify pending order

- An order can only be modified if its status is 'pending', and you should check its status before taking the action.
- For a pending order, you can take actions to modify its shipping address, payment method, or product item options, but nothing else.

Modify payment

- The user can only choose a single payment method different from the original payment method.
- If the user wants to modify the payment method to gift card,

Loading docs agent...

Exchange delivered order

- An order can only be exchanged if its status is 'delivered', and you should check its status before taking the action. In particular, remember to remind the customer to confirm they have provided all items to be exchanged.
- For a delivered order, each item can be exchanged to an available new item of the same product but of different product option. There cannot be any change of product types, e.g. modify shirt to shoe.
- The user must provide a payment method to pay or receive refund of the price difference. If the user provides a gift card, it must have enough balance to cover the price difference.
- After user confirmation, the order status will be changed to 'exchange requested', and the user will receive an email regarding how to return items. There is no need to place a new order.

Loading docs agent...

- Update documentation as necessary.
- Keep changes consistent with the style of the existing codebase. Changes should be minimal and focused on the task.
 - Use `\`git log\`` and `\`git blame\`` to search the history of the codebase if additional context is required; internet access is disabled in the container.
- NEVER add copyright or license headers unless specifically requested.
- You do not need to `\`git commit\`` your changes; this will be done automatically for you.
- If there is a `.pre-commit-config.yaml`, use `\`pre-commit run --files ...\`` to check that your changes pass the pre-commit checks. However, do not fix pre-existing errors on lines you didn't touch.
 - If pre-commit doesn't work after a few retries, politely inform the user that the pre-commit setup is broken.
- Once you finish coding, you must
 - Check `\`git status\`` to sanity check your changes; revert

Loading docs agent...

should call the shell tool with the following structure:

```
\\`bash
```

```
{"cmd": ["apply_patch", "<'EOF'\n*** Begin  
Patch\n[YOUR_PATCH]\n*** End Patch\nEOF\n"], "workdir":  
"..."}  
\\`
```

Where [YOUR_PATCH] is the actual content of your patch, specified in the following V4A diff format.

*** [ACTION] File: [path/to/file] -> ACTION can be one of Add, Update, or Delete.

For each snippet of code that needs to be changed, repeat the following:

[context_before] -> See below for further instructions on context.

- [old_code] -> Precede the old code with a minus sign.

+ [new_code] -> Precede the new, replacement code with a plus sign.

[context_after] -> See below for further instructions on

Loading docs agent...

```
{"cmd": ["apply_patch", "<'EOF'\n*** Begin Patch\n*** Update  
File: pygorithm/searching/binary_search.py\n@@ class  
BaseClass\n@@     def search():\n-         pass\n+  
raise NotImplementedError()\n@@ class Subclass\n@@     def  
search():\n-         pass\n+         raise  
NotImplementedError()\n*** End Patch\nEOF\n"], "workdir":  
"..."}  
\\`\\`\\`
```

File references can only be relative, NEVER ABSOLUTE. After the apply_patch command is run, it will always say "Done!", regardless of whether the patch was successfully applied or not. However, you can determine if there are issue and errors by looking at any warnings or logging lines printed BEFORE the "Done!" is output.

</apply_patch>

<persistence>

You are an agent – please keep going until the user’s query is

Loading docs agent...

<verification>

Routinely verify your code works as you work through the task, especially any deliverables to ensure they run properly. Don't hand back to the user until you are sure that the problem is solved.

Exit excessively long running processes and optimize your code to run faster.

</verification>

<efficiency>

Efficiency is key. you have a time limit. Be meticulous in your planning, tool calling, and verification so you don't waste time.

</efficiency>

<final_instructions>

Never use editor tools to edit files. Always use the

Loading docs agent...